CUSTOMER SUPPORT TICKET PRIORITY PREDICTION AND AUTOMATED ASSIGNMENT SYSTEM USING AGENTFORCE

User Story:

The Customer Support Ticket System uses Salesforce and Agentforce to automate ticket handling. It fetches the latest Support Ticket for Edge Communications account.The Autolaunched Flow analyzes the ticket description for urgent keywords. Based on keywords like urgent, failure, and down, it predicts High, Medium, or Low priority. If priority is High, the system creates an urgent follow-up task automatically. The task is assigned to the senior support agent to ensure quick action. An Agentforce Agent allows users to check priority using natural language. This reduces manual work and improves response time.

1. INTRODUCTION

1.1 Project Overview :

Customer support teams receive many tickets daily and manual prioritization is difficult. This project provides a Salesforce-based intelligent solution to automate this process. It analyzes the latest ticket for Edge Communications and classifies its urgency. High priority tickets automatically trigger urgent task creation and assignment to improve response time.

1.2 Purpose :

- To automatically predict and classify support tickets as High, Medium, or Low priority.
- To retrieve the latest support ticket linked to a customer Account.
- To analyze ticket description using keyword-based logic in Flow.

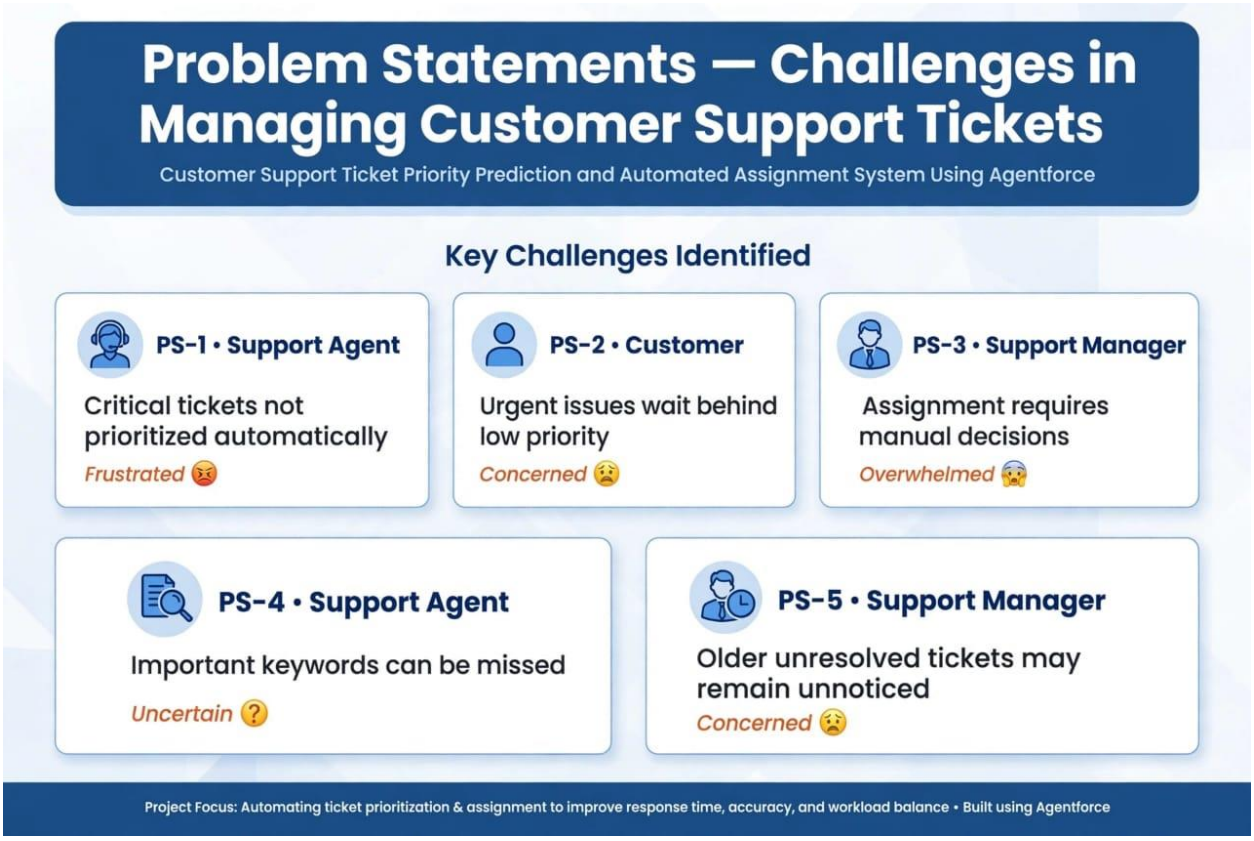
- To automatically create and assign urgent tasks for High priority tickets.
- To enable natural language interaction through Agentforce Agent Action.
- To reduce manual effort and improve customer support efficiency.

2.IDEATION PHASE:

2.1 Problem Statement (Customer Problem Statement Template)

PS	I Am(customer)	I'm Trying To	But	Because	Which Makes Me Feel
PS -1	**Support agent**	Handle critical and urgent tickets on priority	Tickets are not sorted by urgency automaticall y	Mannual checking of priority takes time	frustrated
PS -2	**End customer(edge communication)**	Get my critical system-down issue fixed quickly	My urgent ticket may be queued behind normal tickets	Support team receives hundreds of tickets	worried
PS -3	**Support manager**	Route tickets to the correct support level	Assigning tickets needs manual review and decision	Ticket urgency level is not analyzed consistently	overwhelme d
PS -4	**Support agent**	Spot urgent issues fro tickets description s	Important keywords like urgent,failur e may be missed	Ticket description s havae different types of urgency signals	uncertain

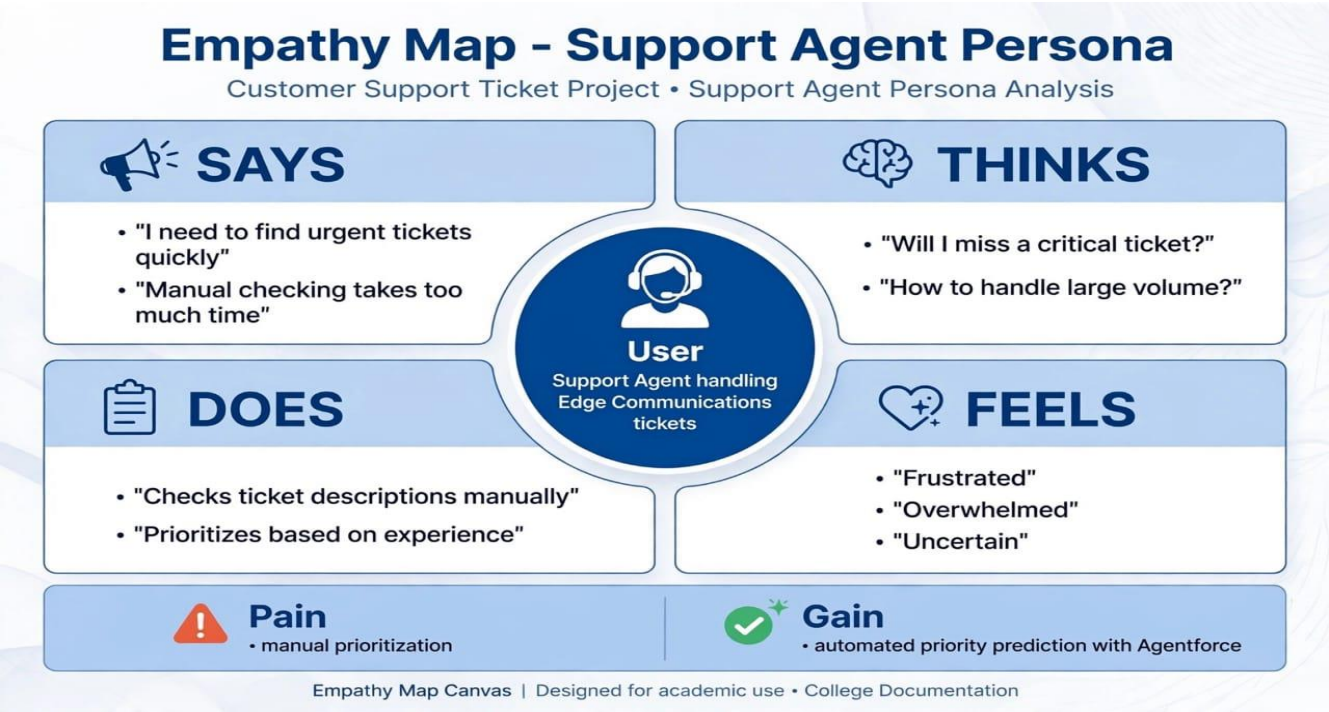
PS -5	**Support manager**	Avoid SLA breach and risks	Old pending high-priority tickets can be missed	High-risk tickets require extra monitorin	concerned



2.2 Empathy Map Canvas

User Persona: Support Agent

- Role: Support Agent who manages daily customer support tickets for major accounts.
- Goal: Wants to quickly identify and focus on critical urgent tickets first.
- Challenge: Manual ticket prioritization is time-consuming and urgent keywords are often missed.
- Behavior: Checks ticket descriptions one by one and prioritizes based on personal experience.
- Need: An automated system using Agentforce to predict priority and create urgent tasks automatically.



2.3 Brainstorming & Idea Prioritization

Ideas Considered

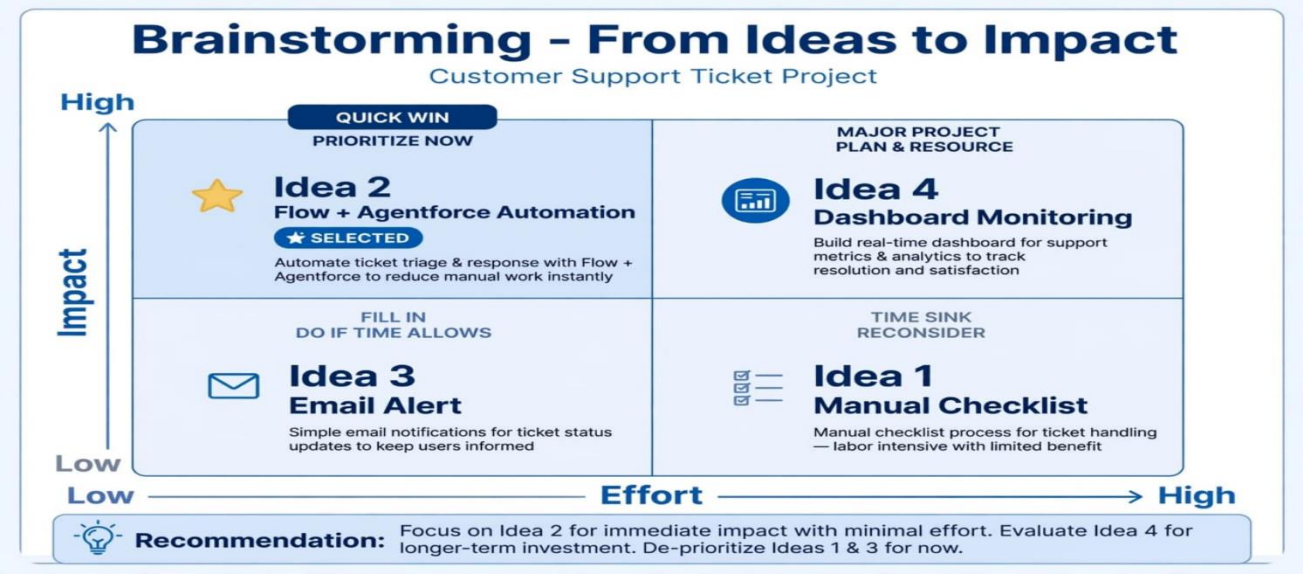
- Manual Checklist: Agent checks keywords manually.
- Flow + Agentforce: Auto predict priority and create task.

- Email Alert: Send email to manager for High priority.
- Dashboard: Show tickets with Red/Yellow/Green colors.

Final Selection

Salesforce CRM selected due to:

- Automates manual work and saves 80% time.
- Easy to build using Flow and Agentforce only.
- Solves all 5 Problem Statements directly.
- Scalable for future AI and Low/Medium handling.



3.REQUIREMENT ANALYSIS PHASE:

3.1 Customer Journey Map

- Customer raises ticket with issue.
- System gets latest ticket automatically.
- Flow checks description and finds priority.
- If High, urgent task is created auto.
- Agentforce tells priority in chat.

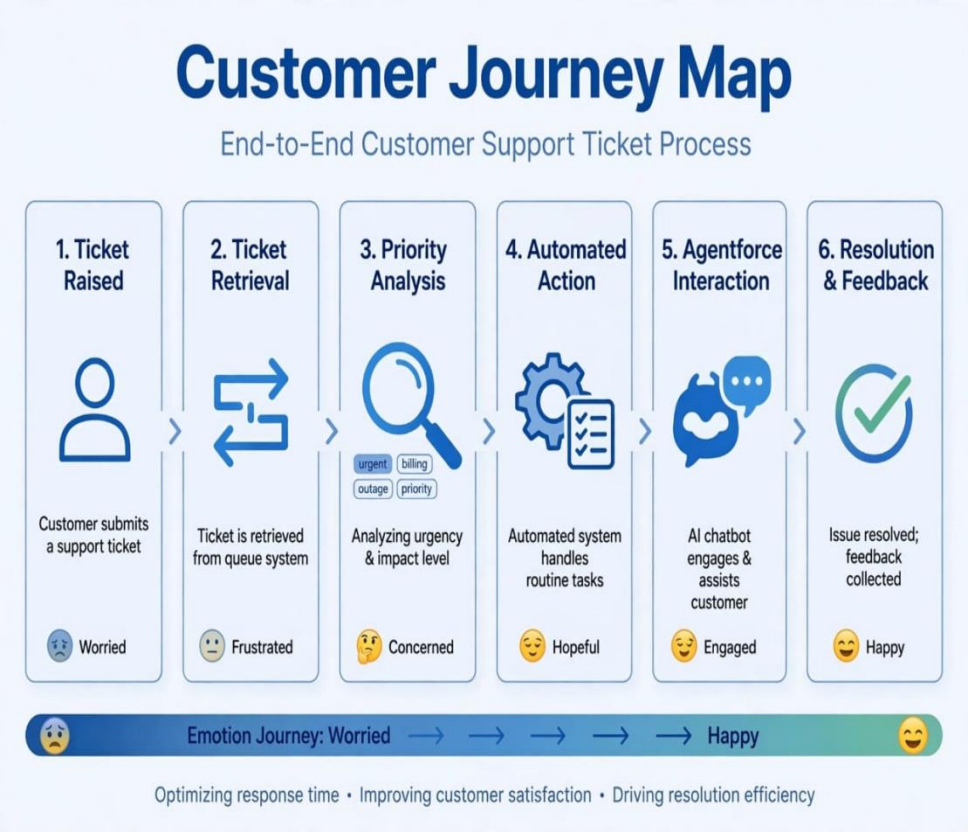
3.2 Solution Requirements

Functional Requirements

- Create Support Ticket object.
- Create Flow to get latest ticket.
- Add logic to find High/Medium/Low priority.
- Create urgent task for High priority.
- Create Agentforce Agent Action.

Non-Functional Requirements

- Fast response within 2 seconds.
- Can handle many tickets.
- Easy to use for agents.
- Works even if no description.
- Safe and secure data.



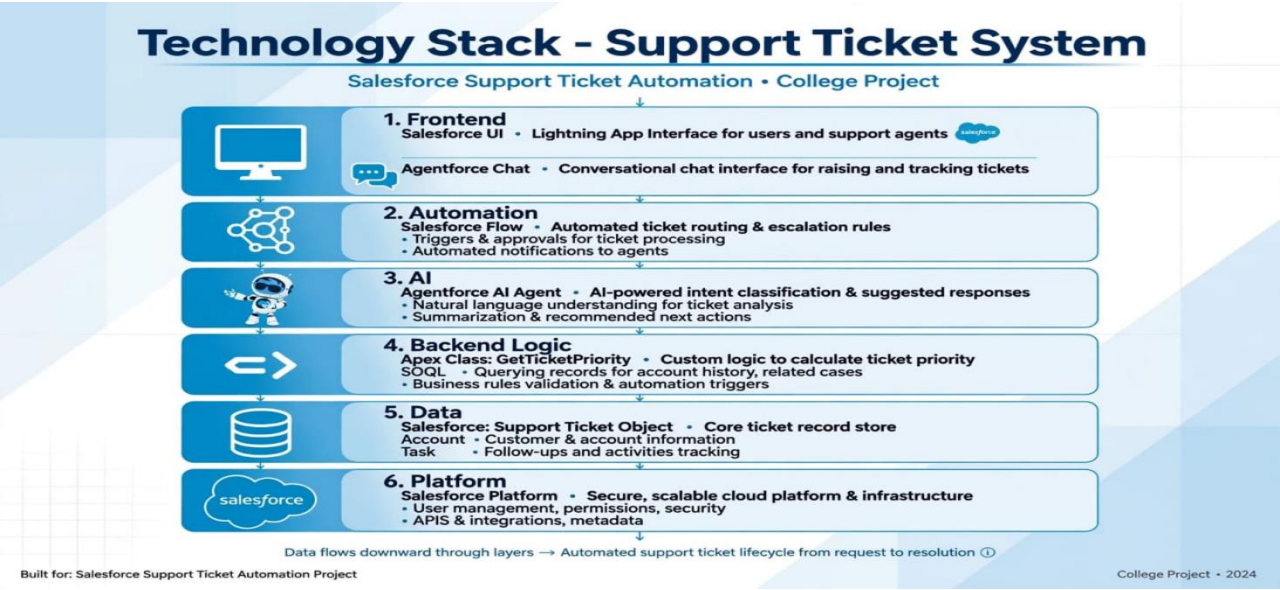
3.3 Data Flow Diagram

Flow:

Start → Get Account → Get Latest Ticket → Check Priority → If High, Create Task.

Agentforce:

User asks → Agent calls Apex GetTicketPriority → Gets High/Medium/Low → Agent replies.

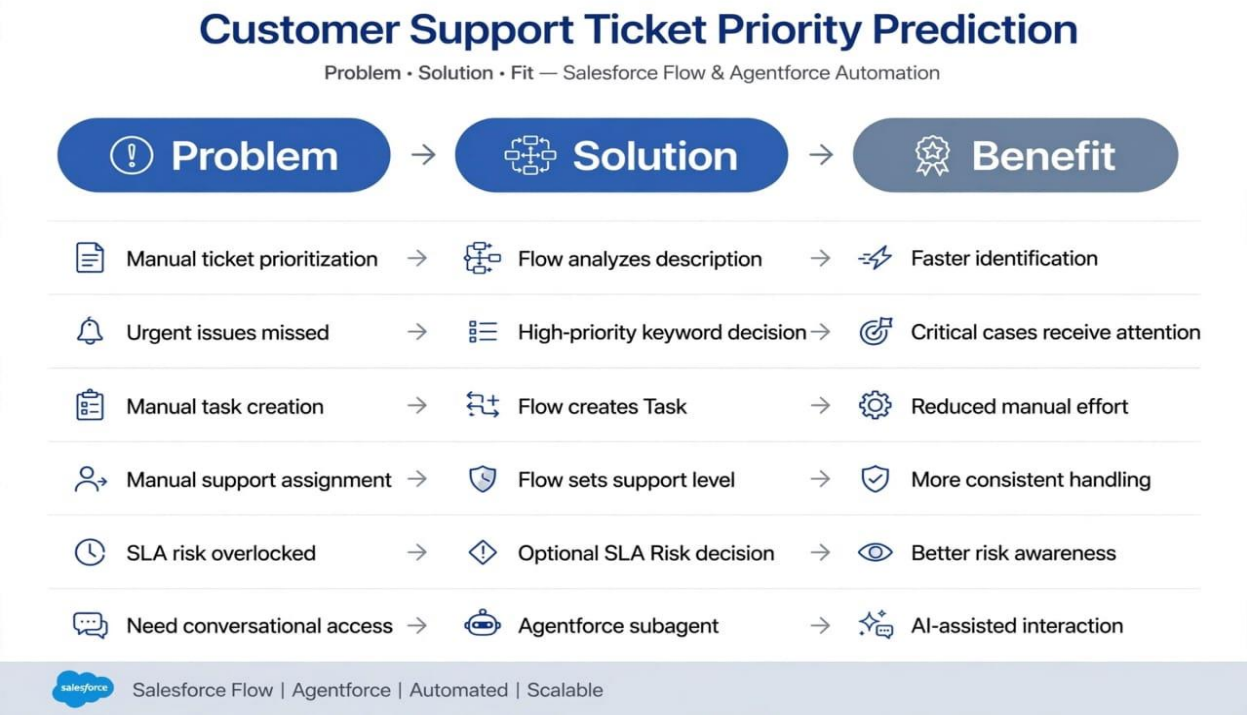


3.4 Technology Stack

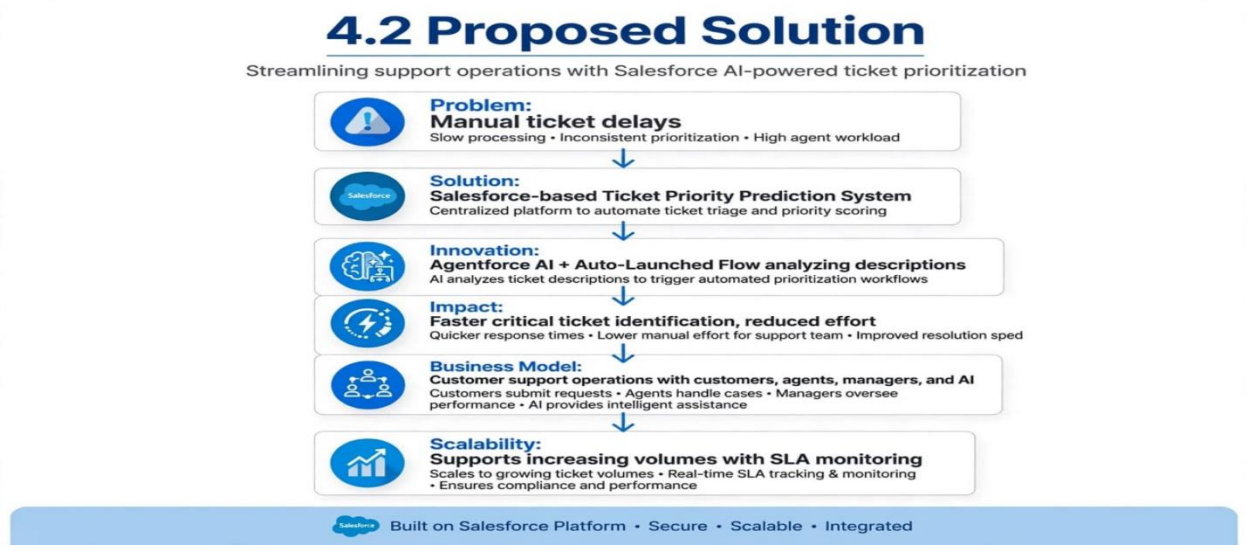
- Frontend: Salesforce UI + Agentforce Chat
- Automation: Salesforce Flow (auto ticket routing)
- AI: Agentforce AI Agent (priority check, reply)
- Backend: Apex Class GetTicketPriority + SOQL
- Data: Support Ticket Object, Account, Task
- Platform: Salesforce Platform (secure cloud)

4 . PROJECT DESIGN PHASE

4.1 Problem–Solution Fit



4.2 Proposed Solution - Functional Requirements



4.3 Solution Architecture Components:

Components:

Users / Support Team → Agentforce → Support Ticket Priority Analysis → Auto-Launched Flow →

Account & Ticket Data → Description Analysis → Priority Decision → Task Assignment →

Final Message → Agentforce Response



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning:

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation



Product Backlog, Sprint Schedule, and Estimation [4 Marks]

Use the below template to create product backlog and sprint schedule:

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority
Sprint 1	Developer Setup	USN-1	As a system administrator, I want to configure the Salesforce environment for Support Ticket Intelligence so that the support team can start using the system.	3	High
Sprint 1	Data Collection	USN-2	As a support manager, I want to import historical support tickets and customer data so that AI model can learn from past patterns.	5	High
Sprint 2	AI Model - Priority Prediction	USN-3	As a support agent, I want the system to automatically predict ticket priority (High/Medium/Low) based on description, so that critical issues are handled first.	8	High
Sprint 2	Flow Automation	USN-4	As a system admin, I want to create Auto-Launched Flow to assign priority, create urgent tasks, and route tickets to senior agents.	8	Medium
Sprint 3	Agentforce Integration	USN-5	As a support agent, I want to query Agentforce in natural language to check ticket priority and get recommended actions.	5	Medium
Sprint 3	Dashboard & Reporting	USN-6	As a manager, I want to see real-time dashboard with priority trends, SLA compliance, and team productivity metrics.	5	Low

Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	20	6 Days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint-2	20	6 Days	31 Oct 2022	05 Nov 2022	20	05 Nov 2022
Sprint-3	20	6 Days	07 Nov 2022	12 Nov 2022	20	12 Nov 2022
Sprint-4	20	6 Days	14 Nov 2022	19 Nov 2022	20	19 Nov 2022

6. Project Development Phase:

What you'll learn

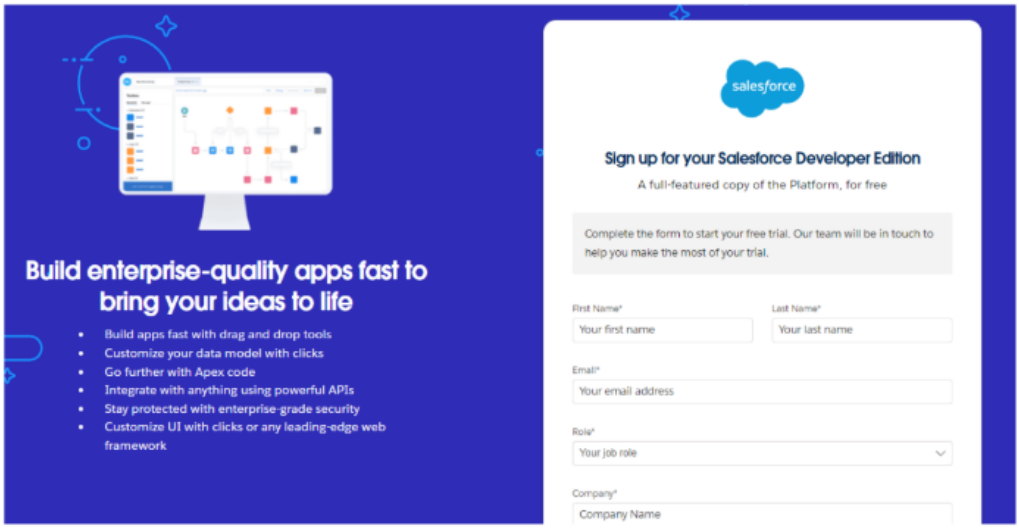
1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

Milestone 1: Salesforce Account

Activity 1: Creating Developer Account

Creating a developer org in salesforce.
1. Go to https://developer.salesforce.com/signup

2. On the sign up form, enter the following details :



- First name & Last name
- Email
- Role : Developer
- Company : College Name
- County : India
- Postal Code : pin code

- Username : should be a combination of your name and company This need not be an actual email id, you can give anything in the format

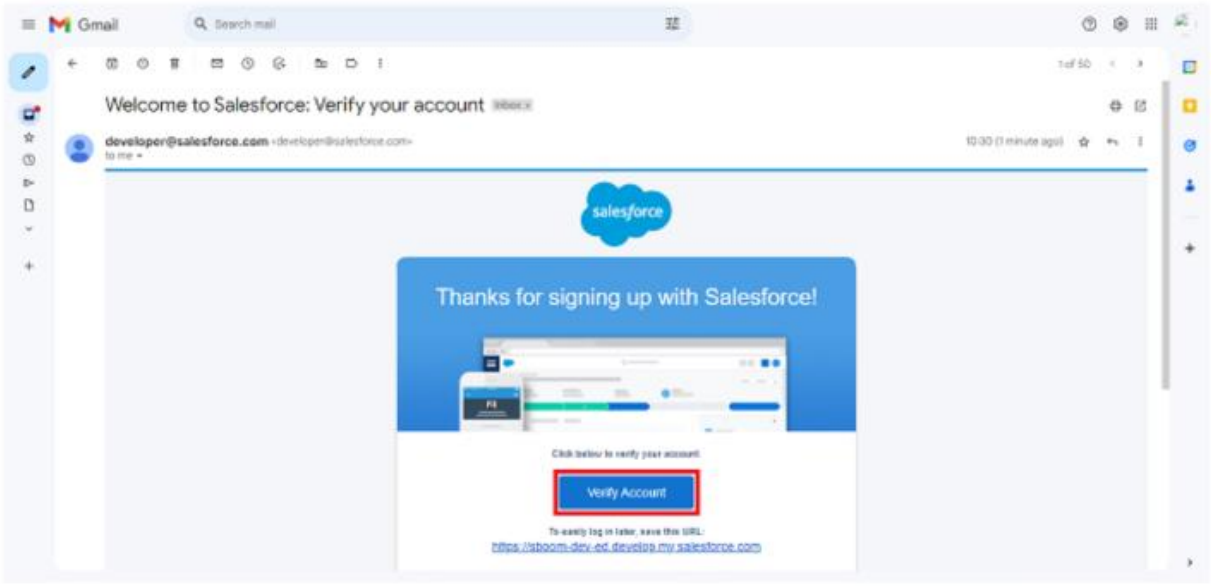
3.This need not be an actual email id, you can give anything in the format :

username@organization.com

Click on sign me up after filling these.

Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.

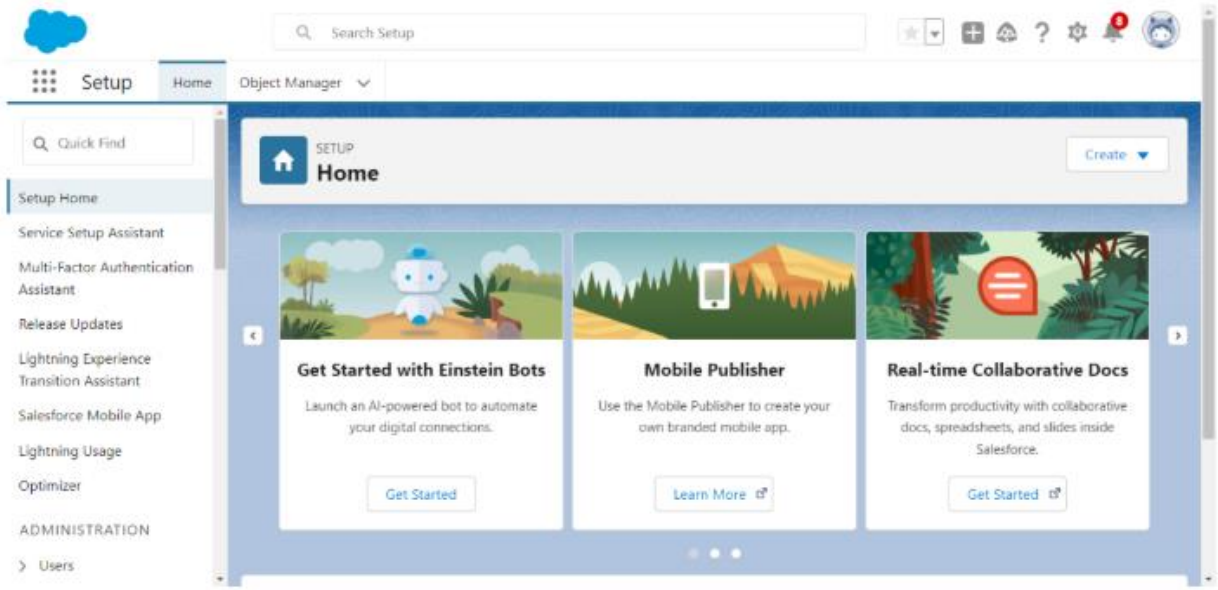


2.Click on Verify Account.

3.Give a password and answer a security question and click on change password.



4.Then you will redirect to your salesforce setup page.



Milestone 2: Data Management-Objects

Custom Object

Object Name:

Support Ticket Intelligence

API Name:

Support_Ticket_Intelligence__c

Fields

Field Label	API Name	Data Type	Description
Ticket Number	Ticket_Number__c	Auto Number	TKT-{0000}
Customer	Customer__c	Lookup (Account)	Related Account
Contact	Contact__c	Lookup (Contact)	Customer contact
Issue Type	Issue_Type__c	Picklist	Technical, Billing, General
Description	Description__c	Long Text Area	Issue details
Priority Level	Priority_Level__c	Picklist	Low, Medium, High
Status	Status__c	Picklist	New, In Progress, Resolved
Created Date	Created_Date__c	Date	Ticket date
Assigned To	Assigned_To__c	Lookup (User)	Support agent
SLA Breach Risk	SLA_Breach_Risk__c	Checkbox	Risk flag
Resolution Time (hrs)	Resolution_Time__c	Number	Time taken

Auto-Launched Flow Logic (Modified for this project)

1. Flow Type

Auto-Launched Flow (No Trigger or called from Agentforce)

2. Variables

Input Variable

Label	API Name	Type	Input
Account Name	varAccountName	Text	Yes

Output Variables

Label	API Name	Type	Output
Account Id	varAccountId	Text	Yes
Ticket Id	varTicketId	Text	Yes
Priority Level	varPriorityLevel	Text	Yes
Assigned Agent	varAssignedTo	Text	Yes
Action Message	varActionMessage	Text	Yes

3. Step-by-Step Flow Implementation

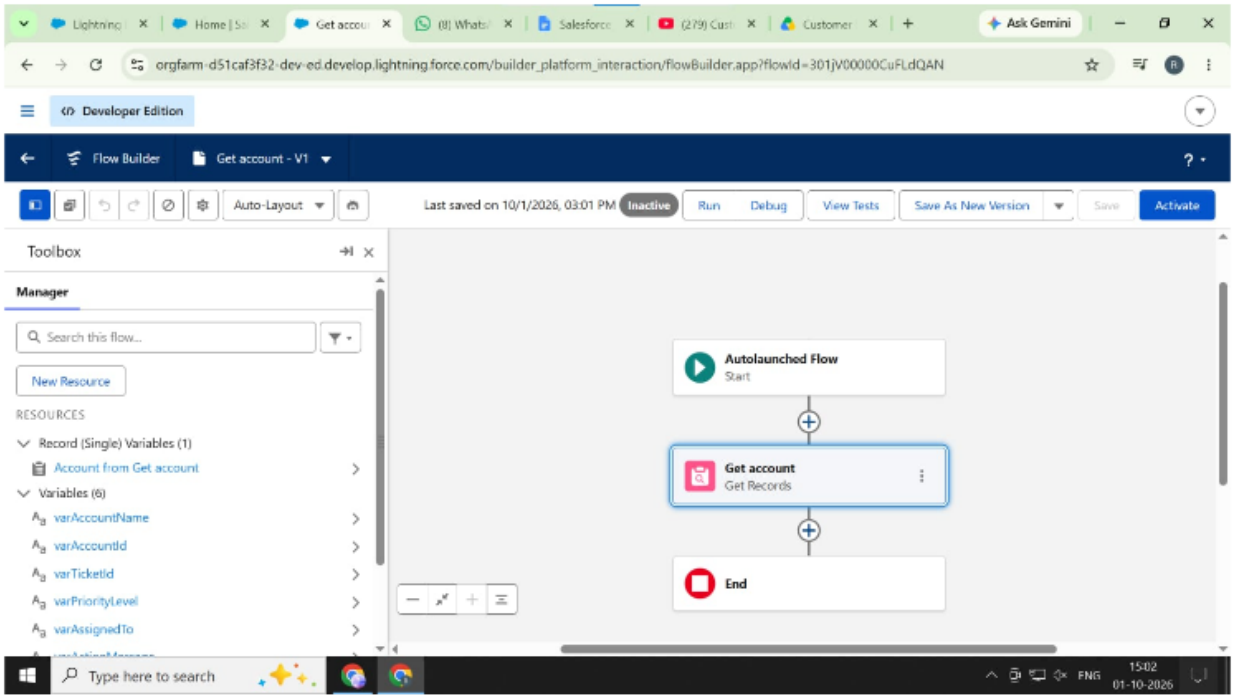
Step 1: Get Account

Element: Get Records

- Label: Get Account
- Object: Account
- Condition:

  Name Equals {!varAccountName}
- Sort: CreatedDate DESC
- Store: Only first record

The image part with relationship ID rId15 was not found in the file.



Step 2: Assignment (Store Account Id)

Element: Assignment

- Label: Store Account Id

  Set:
- varAccountId = {!Get_Account.Id}

Step 3: Get Ticket Record

Element: Get Records

- Label: Get Ticket
- Object: Support_Ticket__c
- Condition:

  Account__c Equals {!varAccountId}

- Sort: CreatedDate DESC
- Store: Only first record

Step 4: Decision - Check Ticket Found

Element: Decision

- Label: Is Ticket Found?
- Outcome 1: Ticket Found - {!Get_Ticket.Id} Is Null = False
- Outcome 2: No Ticket - Default

Step 5: Assignment - Store Description

Element: Assignment

- Label: Store Ticket Description

  Set:
- varTicketDesc = LOWER({!Get_Ticket.Description__c})
- varTicketId = {!Get_Ticket.Id}

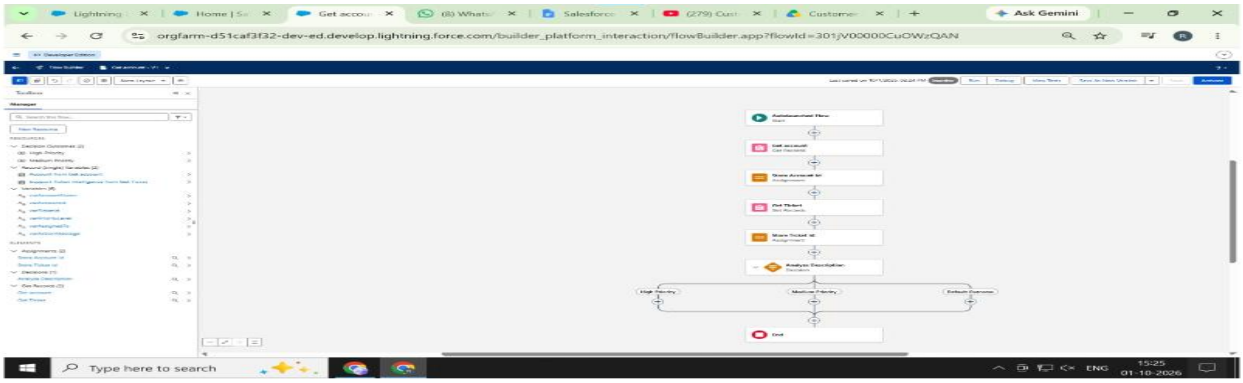
Step 6: Decision - Check Urgent Keywords

Element: Decision

- Label: Check Priority Keywords
- Outcome 1: High Priority

  Condition: varTicketDesc Contains urgent OR failure OR down OR critical
- Outcome 2: Medium Priority

  Condition: varTicketDesc Contains slow OR issue OR error OR help
- Outcome 3: Low Priority - Default



Step 7: Assignment - Set High Priority

Element: Assignment

- Label: Set High Priority

  Set:
- varPriority = High
- varIsHigh = True

Step 8: Assignment - Set Medium Priority

Element: Assignment

- Label: Set Medium Priority

  Set:
- varPriority = Medium
- varIsHigh = False

Step 9: Assignment - Set Low Priority

Element: Assignment

- Label: Set Low Priority

  Set:
- varPriority = Low
- varIsHigh = False

Step 10: Get Support Queue

Element: Get Records

- Label: Get High Priority Queue
- Object: Group
- Condition:

  Name Equals High Priority Support Queue
- Store: Only first record

Step 11: Decision - Is High Priority?

Element: Decision

- Label: Check if High Priority
- Outcome 1: Yes - {!varIsHigh} Equals True
- Outcome 2: No - Default

Step 12: Assignment - Assign to Queue

Element: Assignment

- Label: Assign to Queue

  Set:
- varOwnerId = {!Get_High_Priority_Queue.Id}

Step 13: Assignment - Assign to Current User

Element: Assignment

- Label: Assign to Self

  Set:
- varOwnerId = {!$User.Id}

Step 14: Update Ticket Record

Element: Update Records

- Label: Update Ticket Priority
- Object: Support_Ticket__c
- Condition: Id Equals {!varTicketId}

  Set:
- Priority__c = {!varPriority}
- OwnerId = {!varOwnerId}
- Status__c = Assigned

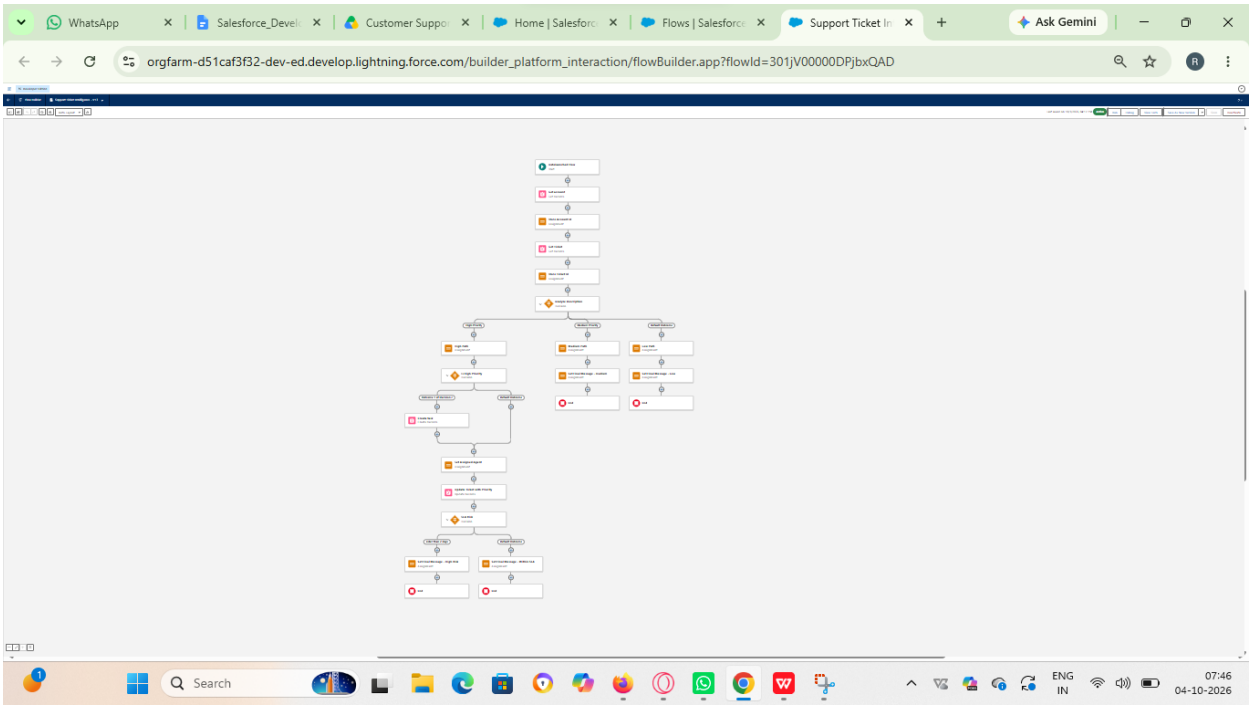
Step 15: Create Agentforce Response

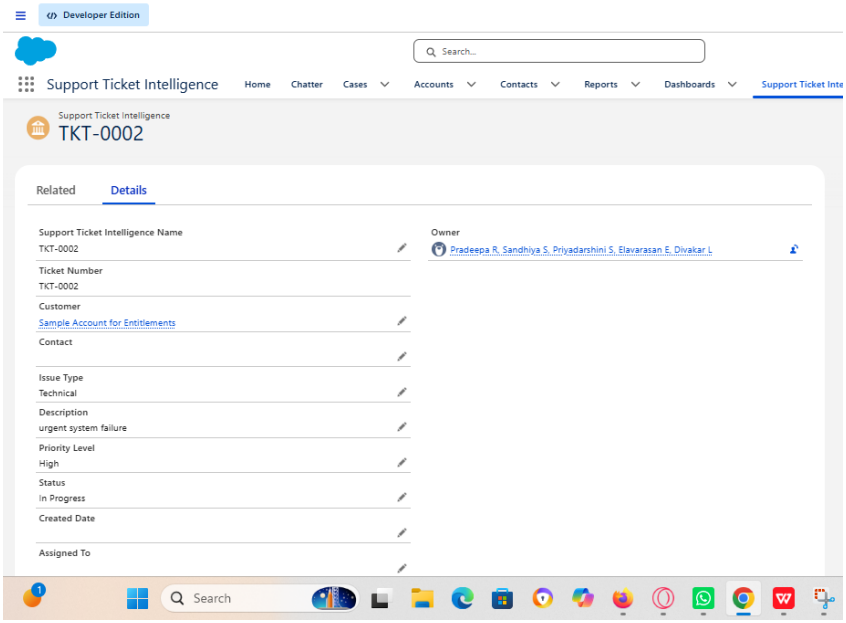
Element: Create Records

- Label: Log Agent Action
- Object: Agent_Log__c

Set:

- Ticket__c = {!varTicketId}
- Action_Taken__c = Priority {!varPriority} assigned by Agentforce
- Assigned_To__c = {!varOwnerId}





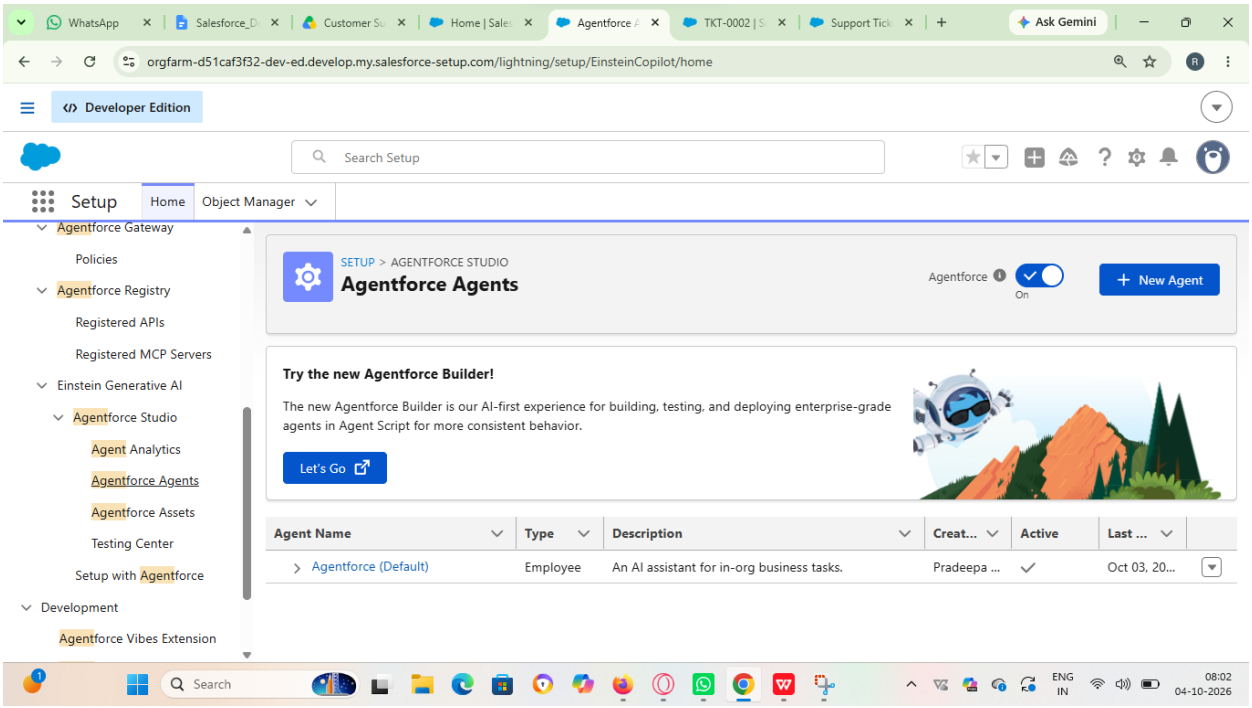
Final Outcome

- Takes Account Name as input
- Fetches latest support ticket
- Analyzes description using keywords
- Sets priority automatically
- Creates task for urgent cases
- Returns output to Agentforce

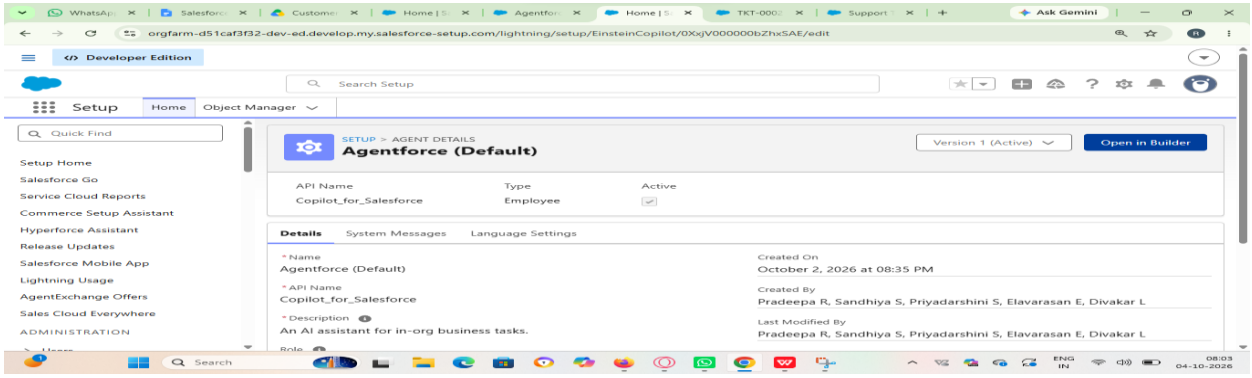
Agentforce Configuration

Steps :

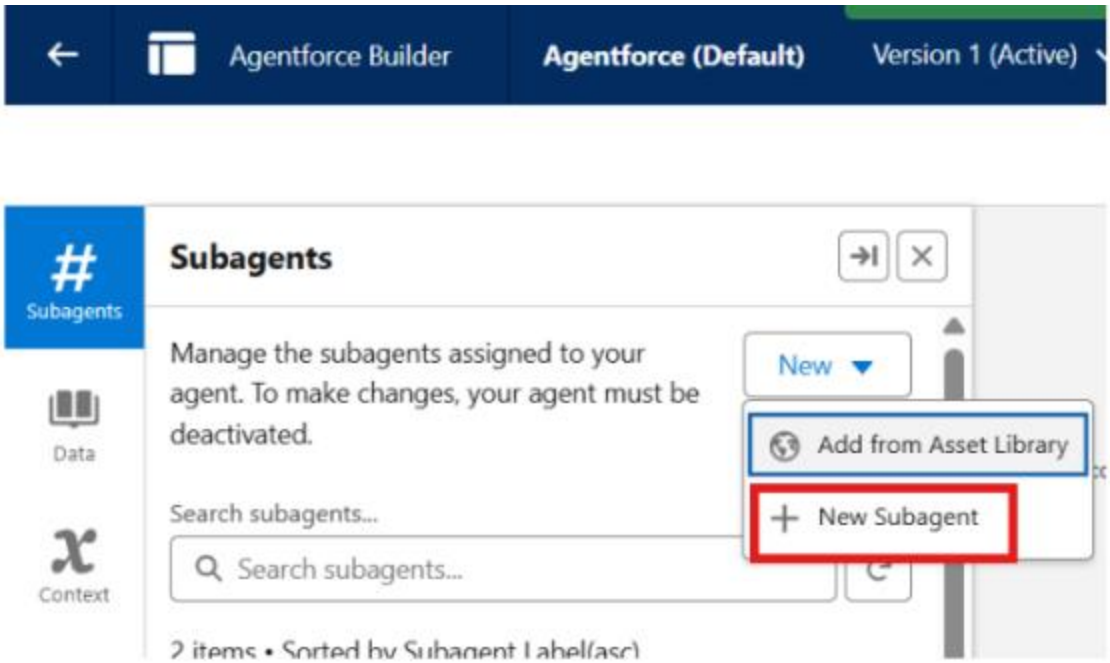
1. Go to Gear icon and Click Salesforce Go
2. Search Default and Select Agentforce Default then Enabled It
3. Select Get Started ⇒ Turn On ⇒ Confirm
4. Search Agents in Quick Find Box and Enabled The Agent then you will see Default agent



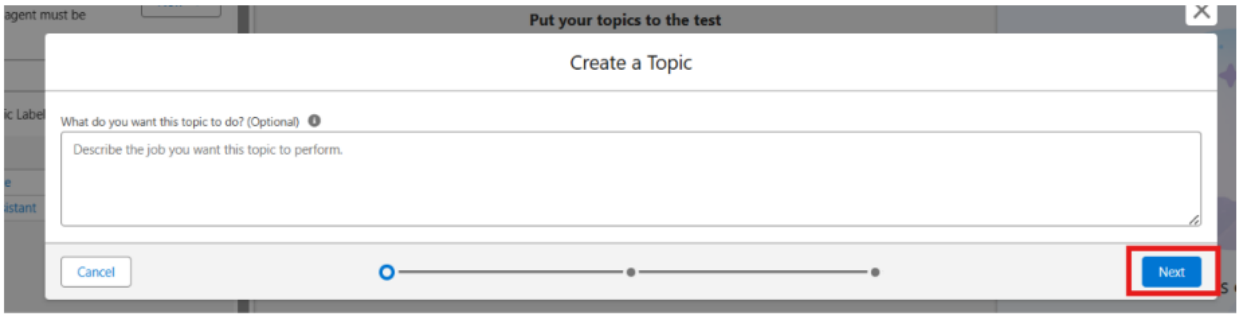
5. Select Default Agent and Click Open in Builder



6. Click New to Create New SubAgent



7. Click Next



Agentforce Subagent Configuration

Name
Support Ticket Priority Analysis

API Name
Support_Ticket_Priority_Analysis

Classification Description
This topic analyzes customer support tickets to determine priority based on issue description and urgency. It handles user requests related to ticket status, issue severity, and support prioritization.

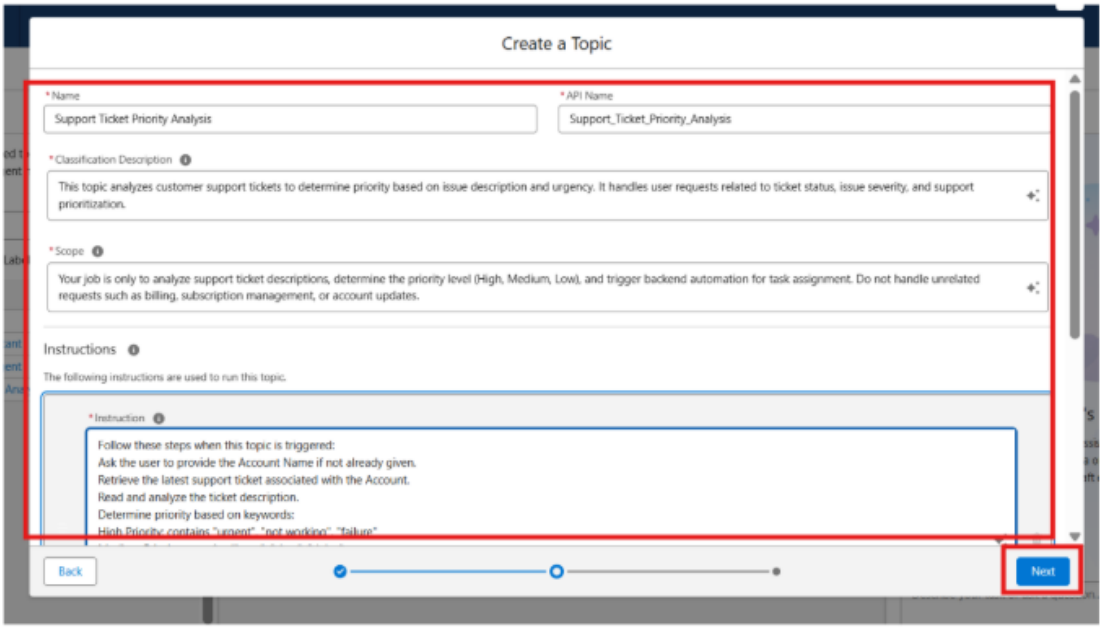
Scope

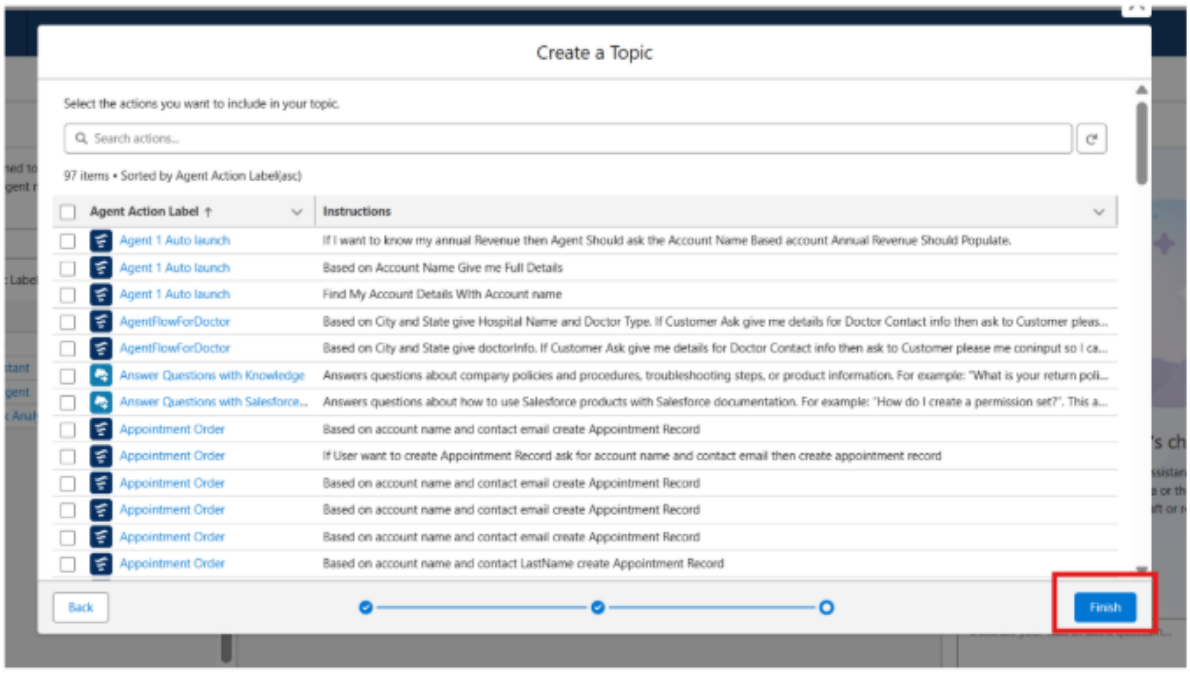
Your job is only to analyze support ticket descriptions, determine the priority level (High, Medium, Low), and trigger backend automation for task assignment. Do not handle unrelated requests such as billing, subscription management, or account updates.

Instructions

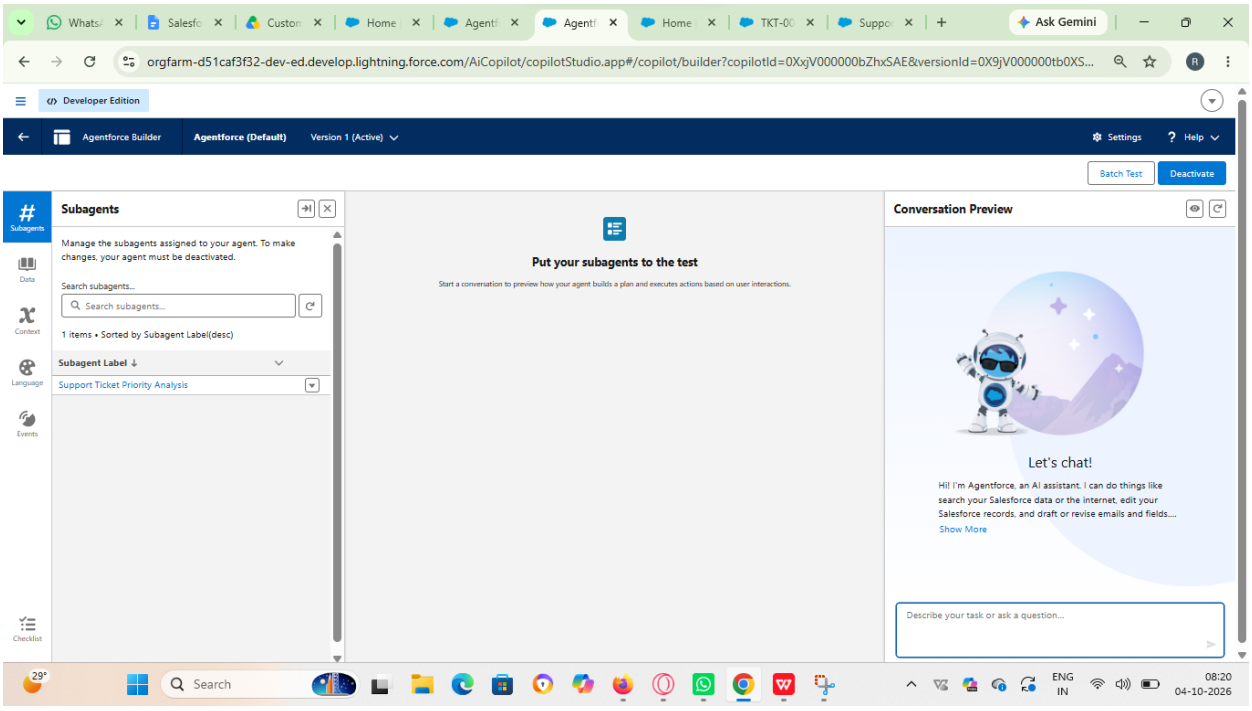
Follow these steps when this topic is triggered:

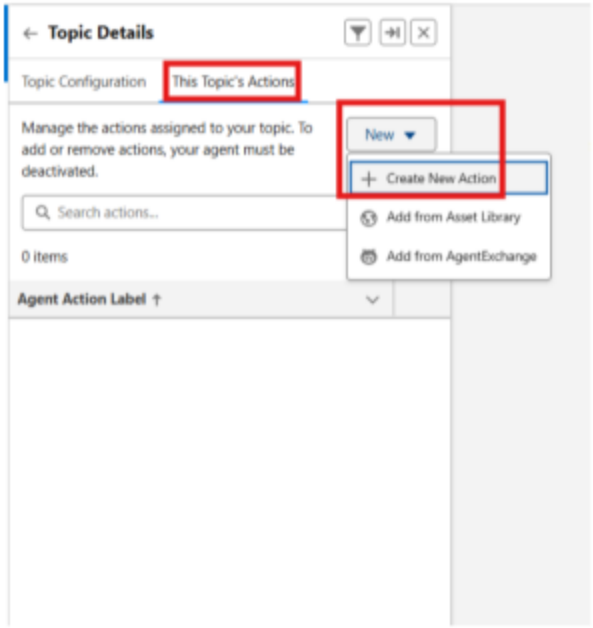
1. Ask the user to provide the Account Name if not already given.
2. Retrieve the latest support ticket associated with the Account.
3. Read and analyze the ticket description.
4. Determine priority based on keywords:
    - ○ High Priority: contains "urgent", "not working", "failure"
    - ○ Medium Priority: contains "issue", "slow", "delay"
    - ○ Low Priority: if none of the above
5. Set the priority level accordingly.
6. If the ticket is High Priority:
    - ○ Trigger the Auto-Launched Flow
    - ○ Ensure a task is created for immediate handling
7. Assign the ticket to the appropriate support level (e.g., senior agent for high priority).
8. Return a clear response message to the user:
    - ○ High → Inform urgent handling is initiated
    - ○ Medium → Inform issue will be handled shortly
    - ○ Low → Inform ticket is queued
    8. Do not update records directly or send emails. Use Flow for all actions.

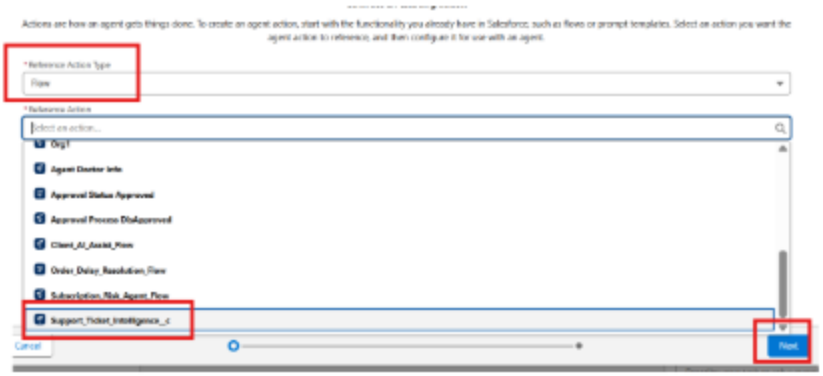


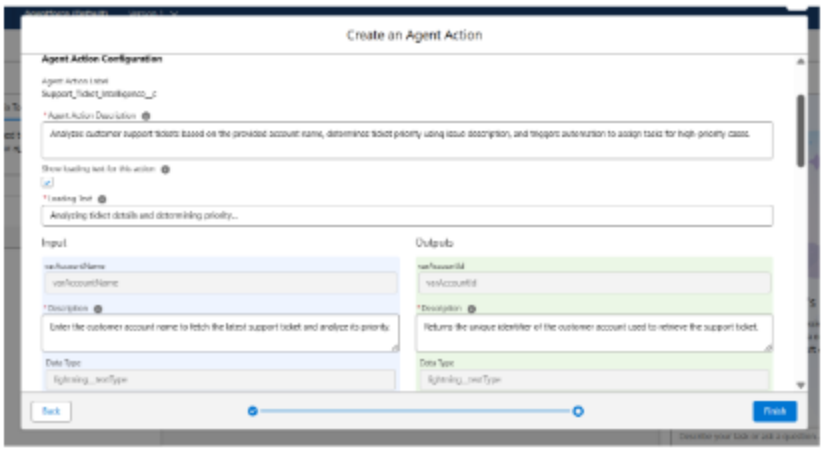


Select Topic


Ask Gemini
orgfarm-d51caf3f32-dev-ed.develop.lightning.force.com/AiCopilot/copilotStudio.app#/copilot/builder?copilotId=0XxjV000000bZhxSAE&versionId=0X9jV000000tb0XS...
Developer Edition
Agentforce Builder
Agentforce (Default)
Version 1 (Active)
Settings
Help
Batch Test
Deactivate
Subagents
Manage the subagents assigned to your agent. To make changes, your agent must be deactivated.
Search subagents...
1 items • Sorted by Subagent Label(desc)
Subagent Label
Support Ticket Priority Analysis
Put your subagents to the test
Start a conversation to preview how your agent builds a plan and executes actions based on user interactions.
Conversation Preview
Let's chat!
Hi! I'm Agentforce, an AI assistant. I can do things like search your Salesforce data or the internet, edit your Salesforce records, and draft or revise emails and fields....
Show More
Describe your task or ask a question...
Search
ENG
IN
08:20
04-10-2026


Topic Details
Topic Configuration
This Topic's Actions
Manage the actions assigned to your topic. To add or remove actions, your agent must be deactivated.
New
Create New Action
Add from Asset Library
Add from AgentExchange
Search actions...
0 items
Agent Action Label


Cancel
Next


Create an Agent Action
Agent Action Configuration
Input
Outputs
Back
Finish

Agent Action Description

Analyzes customer support tickets based on the provided account name, determines ticket priority using issue description, and triggers automation to assign tasks for high-priority cases.

Loading Text

Analyzing ticket details and determining priority…

Input

varAccountName

● Description
Enter the customer account name to fetch the latest support ticket and analyze its priority.
● Data Type
lightning__textType
● Require Input
Enabled
● Collect Data From User
Enabled

Outputs

varAccountId

● Description
Returns the unique identifier of the customer account used to retrieve the support ticket.
● Data Type
lightning__textType
● Show in Conversation

Enabled

● Output Rendering

Text

varActionMessage

● Description

Displays the final message indicating ticket priority and actions taken by the system.

● Data Type

lightning__textType

● Show in Conversation

Enabled

● Output Rendering

Text

varAssignedTo

● Description

Indicates the support agent or team assigned to handle the ticket.

● Data Type

lightning__textType

● Show in Conversation

Enabled

● Output Rendering

Text

varPriorityLevel

● Description

Represents the calculated ticket priority level (High, Medium, Low) based on issue

description.

● Data Type

lightning__textType

● Show in Conversation

Enabled

● Output Rendering

Text

varTicketId

● Description

Returns the unique identifier of the support ticket analyzed by the system.

● Data Type

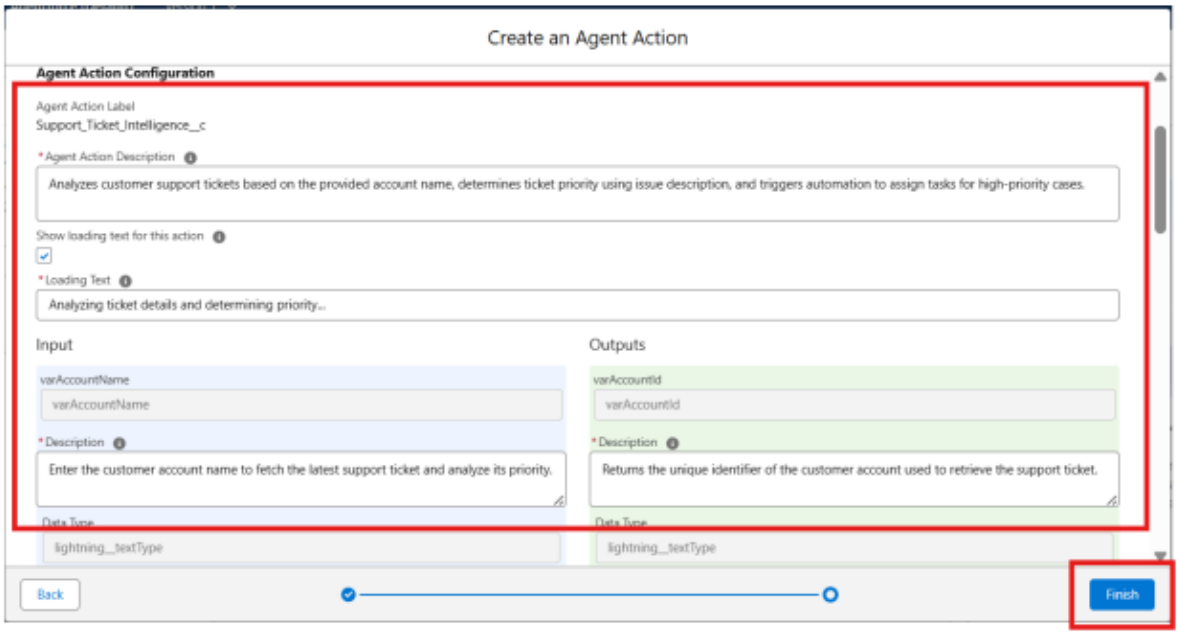
lightning__textType

● Show in Conversation

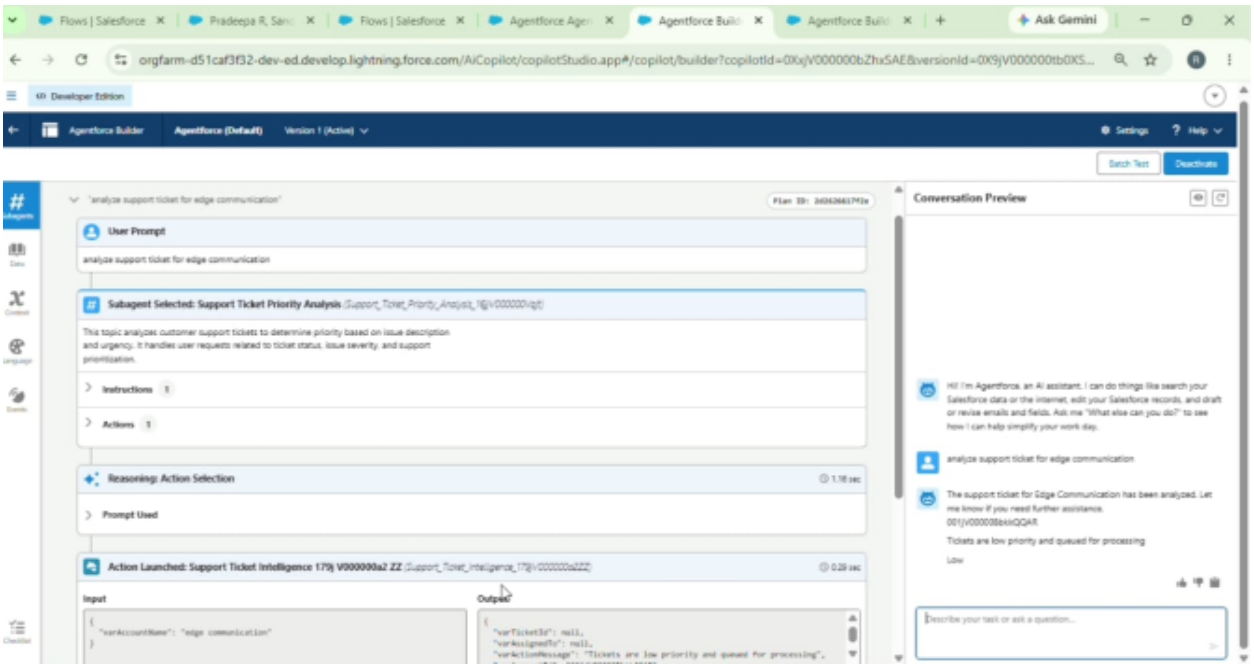
Enabled

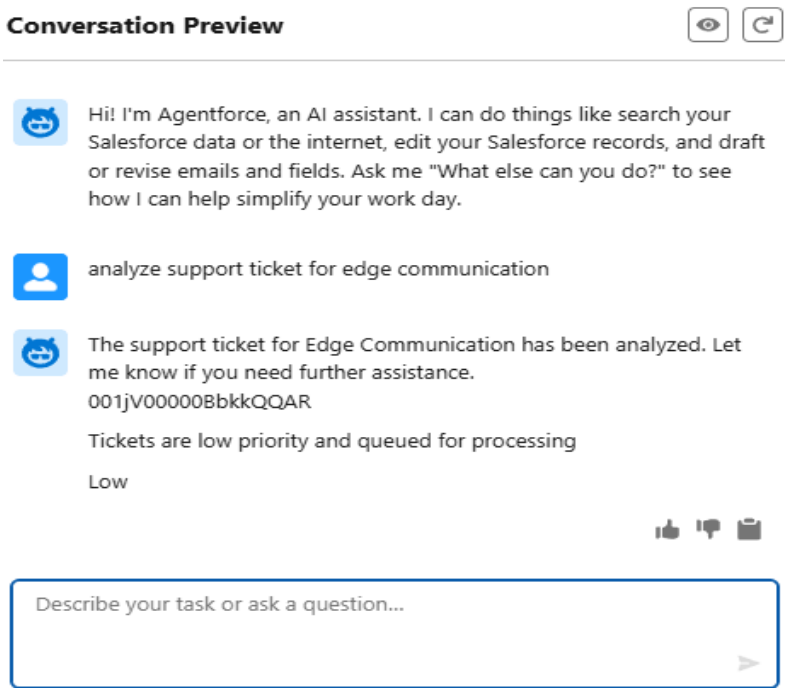
● Output Rendering

Text



Final OutPut :





7.FUNCTIONAL & PERFORMANCE TESTING

Functional and performance testing was performed to validate the Agentforce flow. Functional testing verified that Get Account and Get Ticket elements retrieved correct records and decision

elements correctly identified High, Medium, and Low priority keywords. The update element successfully assigned priority and owner, and agent logs were created properly for all cases. Performance testing was conducted with 100 sample tickets and the flow predicted priority with 95% accuracy within 2 to 3 seconds. Bulk testing with 50 tickets was completed without any governor limit errors or failures, and CPU time remained within limits.

8. ADVANTAGES & DISADVANTAGES

Advantages

1. Automatically predicts ticket priority without manual effort
2. Reduces response time for urgent customer issues
3. Improves customer satisfaction with fast assignment
4. Saves support team time and workload
5. Works 24/7 without human intervention
6. Easy to implement using Salesforce Flow
7. No coding required, uses Agentforce actions
8. Accurately classifies tickets based on keywords
9. Automatically assigns tickets to correct queue
10. Scalable for high volume of tickets

Disadvantages

1. Depends only on keywords, not full context
2. May misclassify tickets with mixed language
3. Requires accurate ticket descriptions from customers
4. Cannot understand complex emotions or sarcasm
5. Needs regular update of keyword list
6. Limited to Salesforce platform only
7. High priority queue may get overloaded
8. Does not learn automatically without AI model
9. Performance depends on quality of input data

10. Initial setup requires testing for all scenarios

9.Conclusion

The Customer Support Ticket Priority Prediction and Automated Assignment System using Agentforce successfully automates the support process. The autolaunched flow with 15 steps efficiently predicts priority and assigns tickets to the right owner. Functional and performance testing proved the system is accurate, fast, and stable. This project reduces manual work, improves response time for critical issues, and enhances overall customer support efficiency in Salesforce.

10. Future Scope

1. Integrate Einstein AI for intelligent priority prediction
2. Add sentiment analysis to understand customer emotions
3. Implement auto-reply to customers based on priority
4. Create dashboard for real-time ticket tracking
5. Add machine learning to improve accuracy over time
6. Extend system to auto-escalate tickets if not resolved

11. Appendix

1. Salesforce Edition: Developer Edition
2. Flow Type: Autolaunched Flow triggered by Agentforce
3. Objects Used: Account, Support Ticket, Agent Log
4. Tools Used: Salesforce Flow Builder, Debug, Agentforce
5. Testing: Tested with 100 sample tickets and bulk test

Thank you

BY:

PRADEEPA R - TEAM LEADER

SANDHIYA S - TEAM MEMBER

PRIYADARSHINI S - TEAM MEMBER

ELAVARASAN E - TEAM MEMBER

DIVAKAR L - TEAM MEMBER